

Rate Limiter

Benefits

- Prevent resource starvation caused by DDOS
- Reduce cost - more important to companies that use third party API = Distributed Denial of Service
- Prevent servers from being overloaded.

Step 1 Understand the problem and Establish design scope.

Q. throttle API requests based on IP, userID, or other?

Q. scale of system?

Reqs :- Limit excessive requests

- Low Latency
- less memory
- Distributed rate limiting
- Exception handling
- High fault tolerance → don't break entire system

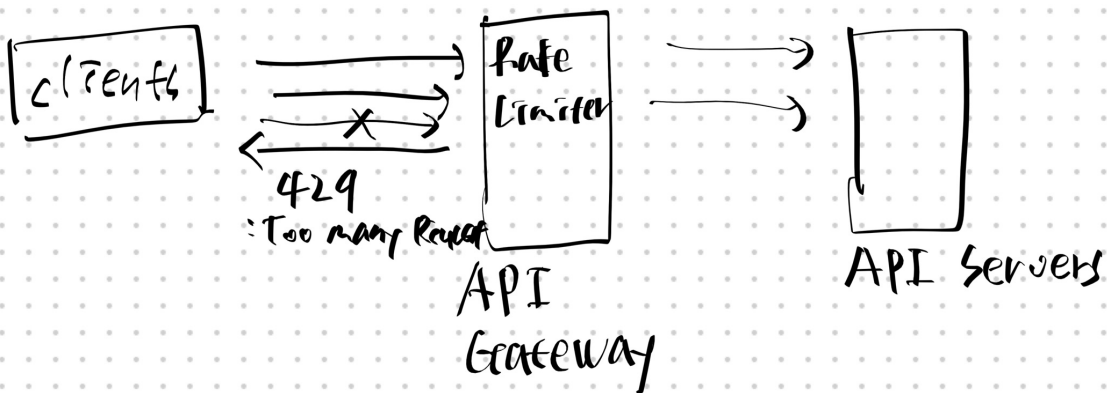
Step 2: Propose high-level design and get buy-in

Where to put?

client-side? → ~~X~~ client requests can easily be forged by many factors.

Impossible to control all clients' situations.

Server-side?



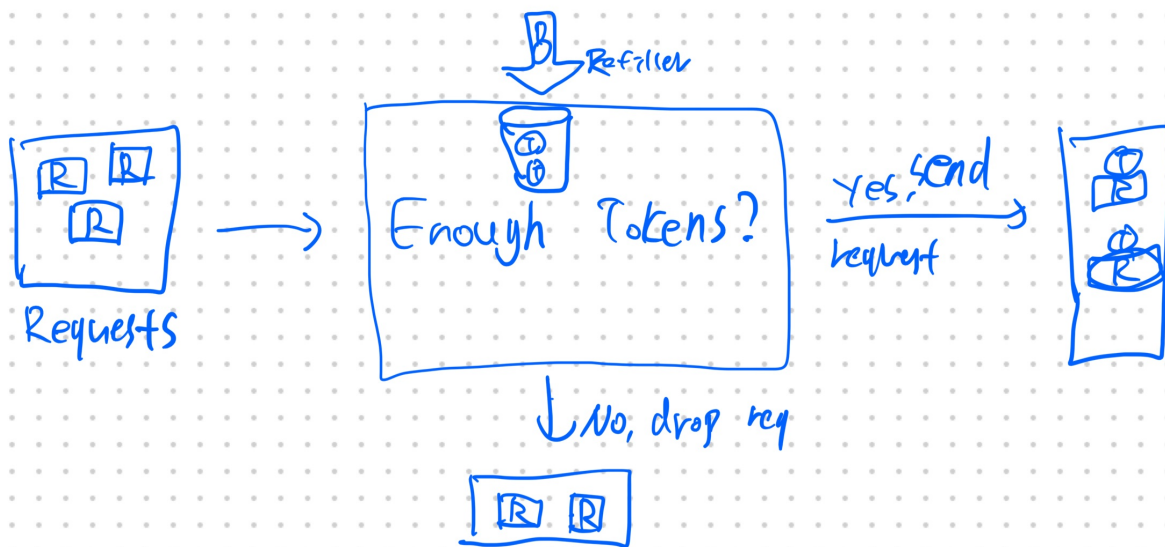
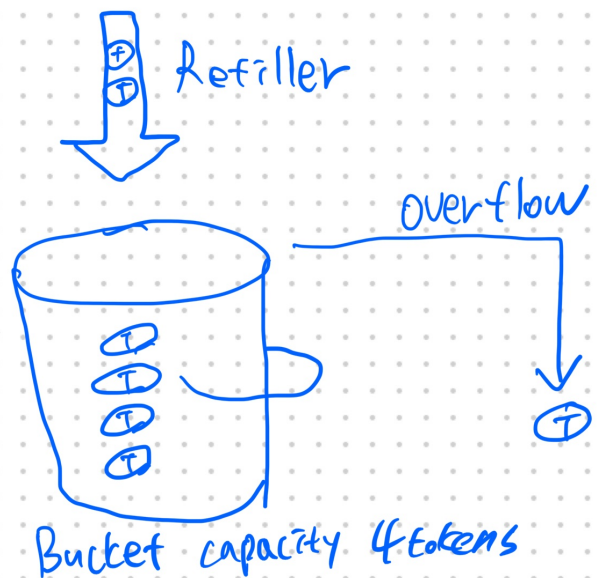
What to consider?

- Technology stack, programming language, cache ...
Check your programming language is efficient to implement a rate limiter.
- Find the best algorithm of rate limiting.
- Micro service and API gateway?
→ good to add this in API gateway.
- If you don't have enough resource?
consider to use commercial API gateway

5 Algorithms for Rate Limiting

① Token bucket

- widely used
- simple, well understood
- Pre-defined bucket $\Rightarrow$ with capacity
- Refiller sends 2 tokens if full, throw out
- A request uses a token



- Bucket size
- Refill Rate

PROS - easy to implement

- Memory efficient

- Allows a traffic burst for short period.

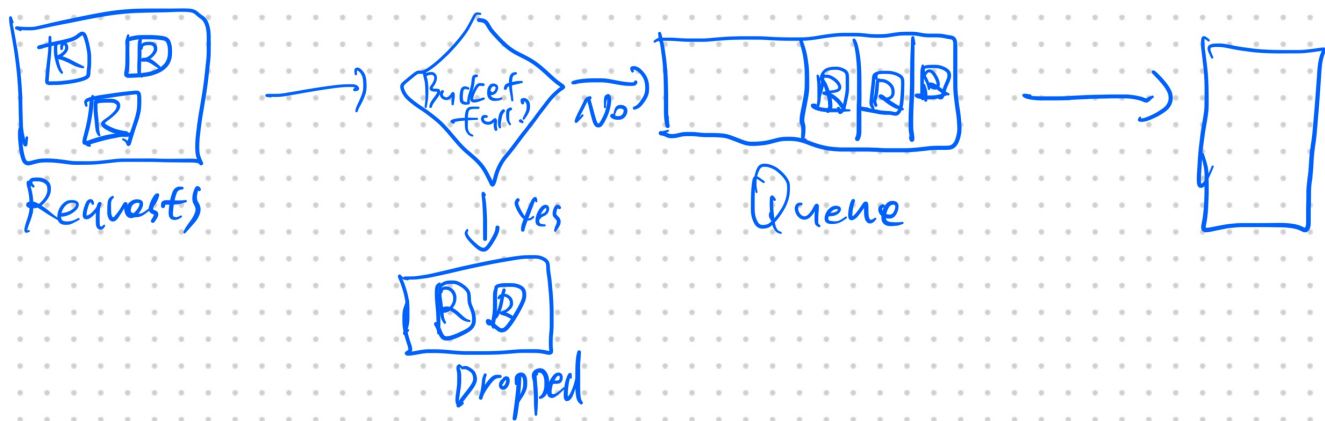
CONS - Not easy to tune

Bucket size and Refiller rate.

② Leaking Bucket Algorithm

FIFO Queue

- check queue is full, if not, add a request into queue
- if yes, drop the request
- Requests are pulled from the queue at regular intervals.



Parameters ① Bucket (Queue) size.

② Outflow Rate: how many Requests can be processed

Pros: - Memory Efficient - given the queue size.
- Stable outflow rate.

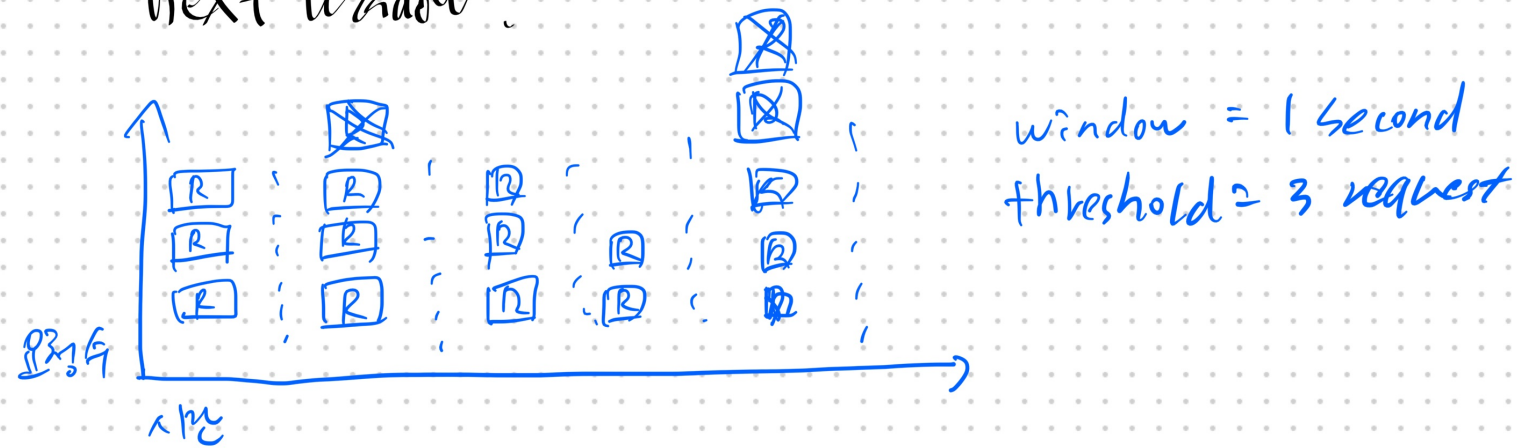
Cons - Queue can have old requests and new request can be dropped.

② Fixed Window Counter Algorithm

타임윈도우 고정된 간격의 윈도우 나누고,
각 윈도우 별로 카운터를 붙인다.

Count += 1 per request

if count $\geq$ threshold : throw requests until
next window.



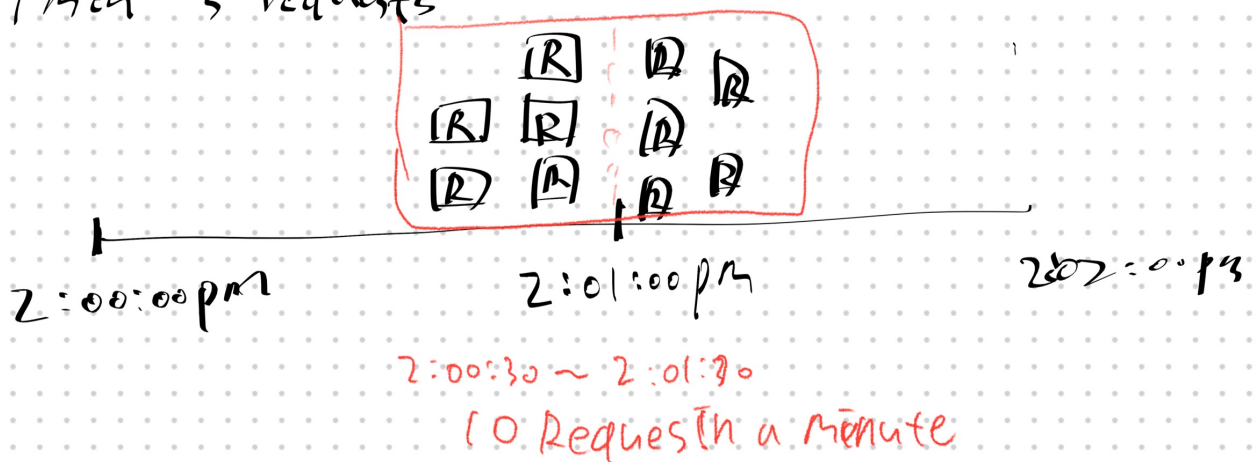
Pros :- Memory efficient

- Easy to understand

- Resetting available quota at the end of
a unit time window fits certain edge cases.

Cons : Spike in traffic at the edge of
a window could cause more requests
than the allowed quota to go through

ex) 1 min 5 requests

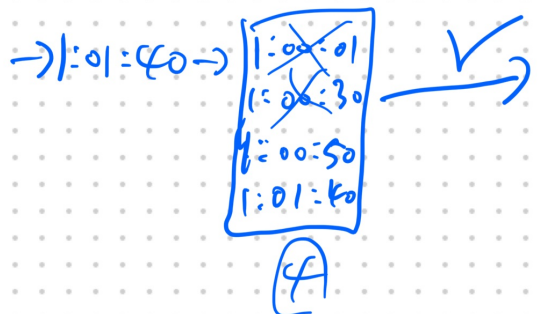
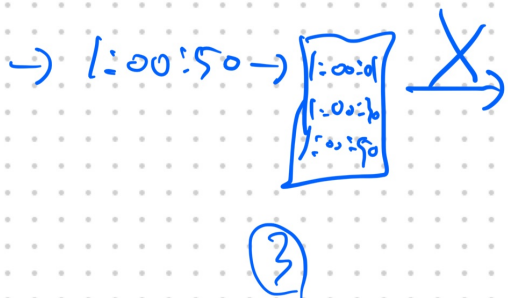
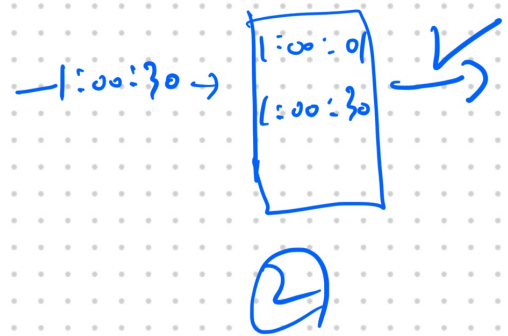
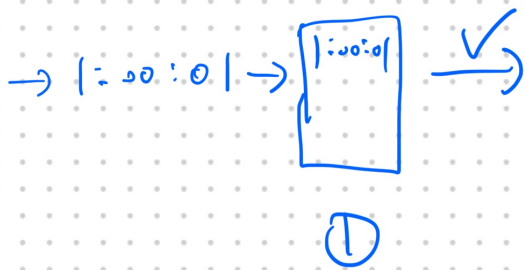


Sliding Window Log Algorithm

this algorithm covers up ③ algorithms problem!

- This algorithm tracks timestamp
↳ saves in Cache (Reds) sorted set
- When a new timestamp receives,
remove old timestamp which is out of window
- add a new time stamp
- if size of the sorted set is over window size,
drops the new request

Allow 2 reqs per minute

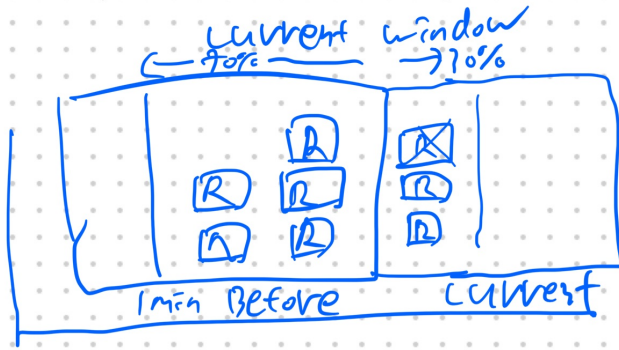


Pros: In any rolling window,
requests will not exceed the rate limit

Cons: consumes a lot of memory,
a rejected request is also saved

⑤ Sliding window Counter Algorithm

= fixed window + sliding window log



7 request per 1 min

PROS: Handle well for a short period traffic based on the average rate of prev window

CONS: It assumes request in the previous window are evenly distributed.
But, it is not a big issue, in real life.